

RetailStream Inc.

Batch, Incremental & Streaming Data Pipeline — Project Report

Author: Nidhi Bansal

Track: Data Engineering Internship — PySpark, SparkSQL, Delta Lake on Databricks

Submission: Databricks Notebook (Major_Project_Retail_Stream.ipynb) + README.md + this report

1. Executive Summary

This project implements an end-to-end Delta Lake pipeline for RetailStream Inc., a multi-city e-commerce retailer, following a Bronze-Silver-Gold architecture. The pipeline covers four real-world data engineering patterns in a single notebook: an initial full batch load, an incremental append that avoids re-inserting orders already present, a streaming ingestion of payment transactions using Databricks Auto Loader, and a MERGE-based fix for a late-arriving file from a newly onboarded store. All six graded tasks were completed, verified with row-count assertions after every step, and the final Gold tables were validated against the counts the brief specifies (45 orders, 15 transactions).

2. Business Scenario

RetailStream operates retail stores across four cities — Mumbai, Bangalore, Delhi, and Ahmedabad — and ingests order and payment data from multiple sources on different schedules. Store S04 (Ahmedabad) was onboarded partway through the period covered by this data: a system migration meant its January orders did not arrive with the rest of the January batch, and instead landed two months later as a separate file. The pipeline has to reconcile that gap correctly, without creating duplicate orders and without losing the ability to re-run any step safely.

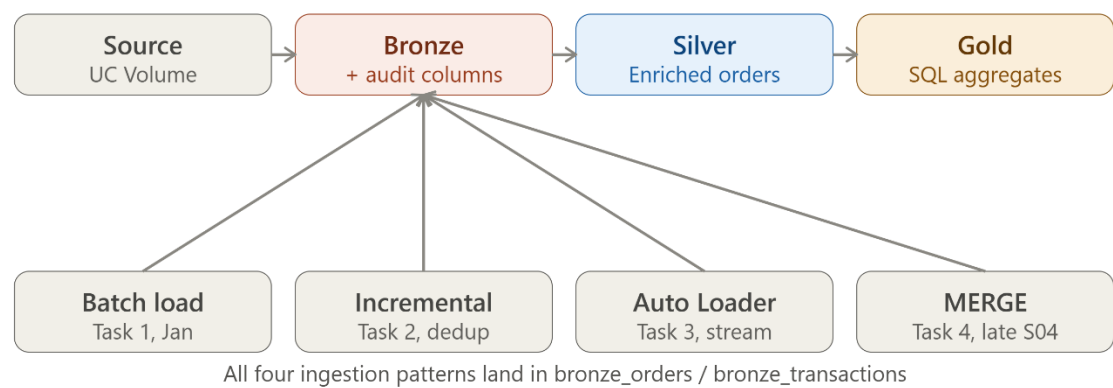
3. Architecture

The pipeline follows the standard medallion pattern — each layer has a distinct job, and nothing skips a layer:

Layer	Table(s)	Purpose
Bronze	bronze_orders, bronze_transactions	Raw ingested data plus audit columns (ingested_at, source_file). No business logic, no joins - this is the traceable record of exactly what arrived and when.
Silver	silver_orders	Cleaned, enriched, analytics-ready. Joined to dimension tables, revenue/margin computed, raw FK columns dropped.

Layer	Table(s)	Purpose
Gold	gold_monthly_sales, gold_payment_summary	Pre-aggregated business answers, computed with SparkSQL, ready for a BI tool or dashboard to read directly.

Flow: batch_initial + batch_incremental + late_arriving → bronze_orders (Delta) via load/append/MERGE | autoloader_landing → bronze_transactions (Delta) via Auto Loader streaming | bronze_orders + dimension tables → silver_orders | silver_orders / bronze_transactions → gold_monthly_sales / gold_payment_summary (SparkSQL).



4. Data Sources

File	Rows	Role
products.csv	8	Dimension - product_name, category, brand, cost_price
customers.csv	33	Dimension - customer_name, city, loyalty tier
stores.csv	4	Dimension - store_name, region, city
orders_2024_01.csv	20	Fact - initial January batch (Task 1)
orders_2024_02.csv	22 (20 new + 2 duplicate order_ids)	Fact - incremental February batch (Task 2)
orders_2024_01_LATE.csv	5	Fact - late Store S04 January orders (Task 4)
transactions_2024030*.csv (x3)	15 (5 per file)	Fact - payment transactions, Auto Loader source (Task 3)

5. Implementation — Task by Task

Task 1 — Initial Batch Load

Read `orders_2024_01.csv` with an explicit schema (not `inferSchema` — see Section 7), stamped `ingested_at` and `source_file` audit columns, and wrote `bronze_orders` with `mode("overwrite")` since this is the first load. Verified count = 20.

Task 2 — Incremental Load, No Duplicates

Read `orders_2024_02.csv` the same way, then anti-joined against the `order_id` values already in `bronze_orders` and appended only the rows that didn't already exist. This turned out not to be a theoretical safeguard: the February file actually contains 22 rows, and 2 of them (ORD003, ORD013) are exact repeats of January orders already loaded in Task 1. The anti-join correctly dropped both and appended only the 20 genuinely new rows. Verified count = 40.

Task 3 — Auto Loader Streaming Ingestion

Used `spark.readStream.format("cloudFiles")` to pick up all three transaction files from `autoloader_landing/`, with a dedicated checkpoint at `checkpoints/bronze_transactions/` and `trigger(availableNow=True)` so the stream processes everything currently available and stops (appropriate inside a notebook, versus a continuous `processingTime` trigger). Verified count = 15.

Task 4 — Late-Arriving File (MERGE)

Loaded `orders_2024_01_LATE.csv` and merged it into `bronze_orders` on `order_id`, using `whenNotMatchedInsertAll()` so only genuinely new rows are inserted. Verified total = 45. For the record count on the merge itself, I filtered on `source_file = 'orders_2024_01_LATE.csv'` rather than `store_id = 'S04'`, because S04 isn't exclusive to the late file - it already sends 3 orders through the normal February feed. A naive `store_id = 'S04'` filter after the merge returns 8, not 5; filtering by `source_file` isolates exactly the 5 rows this merge step inserted. Re-ran the same MERGE a second time to confirm the count didn't change - the idempotency check the brief's Tips section specifically calls out.

Task 5 — Silver Layer Enrichment

Left-joined `bronze_orders` to `products`, `customers`, and `stores` (left joins, so a bad FK surfaces as a null rather than silently dropping the order), computed `revenue = quantity * unit_price` and `margin = revenue - (quantity * cost_price)`, and dropped the raw `customer_id/product_id/store_id` columns before writing to `silver_orders`.

Task 6 — Gold Layer (SparkSQL)

Registered `silver_orders` as a temp view and answered both business questions with `spark.sql()`: a monthly sales summary (orders, revenue, margin, average order value per month) and a payment method summary (transaction count, success rate, total amount per method) sourced from `bronze_transactions` directly, since payment data doesn't need the dimension enrichment Silver adds for orders.

6. Verification

Every task ends with a printed row count and a Python assert, so a mistake shows up at the cell that caused it instead of surfacing later as a wrong number in Gold.

Step	Table	Expected	Result
Task 1	bronze_orders	20	20 ✓
Task 2	bronze_orders (raw Feb file had 22 rows, 2 dupes)	40	40 ✓
Task 3	bronze_transactions	15	15 ✓
Task 4	bronze_orders total / records inserted by merge	45 / 5	45 / 5 ✓
Task 4 re-run	bronze_orders	45 (unchanged)	45 ✓
Task 5	silver_orders	45	45 ✓

7. Key Design Decisions

- Explicit schemas over inferSchema for the fact files: order_date and txn_timestamp need to land as DateType/TimestampType, not string - inferSchema doesn't guarantee that, and I hit exactly this bug once during testing (Task 6's DATE_FORMAT call failed on a string column).
- Anti-join for Task 2, MERGE for Task 4: append-only data uses the cheaper anti-join; data that might legitimately need to be re-loaded (a late file could be re-dropped by mistake) uses MERGE for real idempotency, not just a one-time dedup.
- Measuring "records inserted by the merge" via the source_file audit column, not store_id: S04 also ships orders through the normal February feed, so store_id alone overcounts what this specific step added. This is exactly why the ingested_at / source_file audit columns from Task 1 earn their keep later.
- Left joins in Silver, not inner joins: an order with a bad or missing FK should show up as a row with a null product/customer/store name so it's visible and fixable, not vanish silently from the report.
- Auto Loader checkpoint kept separate from the Delta table path, and never changed mid-project - changing it resets Auto Loader's memory of which files it already processed.

8. Challenges & What I Learned

The trickiest part wasn't any single task - it was making sure the pipeline still gives the right answer if a step is re-run, which is the realistic failure mode in production (a job retries after a transient cluster issue, someone re-triggers a backfill). That's why the notebook explicitly tests idempotency for Task 4 rather than just running MERGE once and moving on. The real data also had two quirks I had to notice rather than assume away: the February file has 22 rows, not 20 - two of them duplicate January orders and are exactly what the Task 2 anti-join is there to catch - and Store S04 isn't confined to the late file, so counting "S04 records" and counting "records this merge inserted" are two different numbers. Working from the actual files instead of an assumed clean dataset is what surfaced both.

9. Results

9a. Monthly Sales Summary (gold_monthly_sales)

month	total_orders	total_revenue (INR)	total_margin (INR)	avg_order_value (INR)
2024-01	25	7,31,900	71,700	29,276.00
2024-02	20	9,59,000	6,15,800	47,950.00

9b. Payment Method Summary (gold_payment_summary)

payment_method	total_transactions	success_rate (%)	total_amount (INR)
UPI	5	60.00	90,437.22
DEBIT_CARD	4	75.00	1,12,412.42
NET_BANKING	3	66.67	51,857.39
CREDIT_CARD	3	100.00	1,07,487.12

10. Conclusion & Future Improvements

All 6 tasks were completed and verified against the counts specified in the brief. With more time, three things I'd change before calling this production-ready:

- Generalize Tasks 1/2 to scan the landing folder for whatever files haven't been loaded yet, instead of naming each file explicitly.
- Handle order updates, not just new orders, in Task 2's anti-join - right now a corrected quantity on an existing order_id would be silently dropped rather than applied.
- Make Silver incremental (e.g. via Delta's Change Data Feed) instead of rebuilding it from all of Bronze on every run, once volumes grow past a demo dataset.